AI for Mathematics

Week 2: Computation, Types, and Dependent Types

Ma Jiajun

School of Math, XMU / Dept. of Math, XMUM

April 16, 2026

Today's Plan

Guiding question:
How do we move from bare computation to a disciplined mathematical language?

Part I
Untyped lambda calculus

Church, Turing, computation, and self-application

Part II
Why types matter

`a : A`, judgments, STLC, and Russell's paradox

Part III
Dependent types

Type families, `Π`, `Σ`, and inductive mathematics

Part I

Untyped Lambda Calculus

Why Start Here?

Before we talk about types, we first ask a simpler question:

What is a computation?

Untyped lambda calculus gives an extremely small language in which computation can already be described.

For Week 2, this is only a starting point.
Untyped lambda calculus is not the final destination; it is the reason typed systems become interesting.

Alonzo Church and the λ-Calculus

Alonzo Church (1903–1995)

- introduced lambda calculus as a formal theory of functions
- gave one of the first rigorous models of computation
- helped shape the modern view that computation can be studied mathematically

Lambda calculus is tiny:

- variables
- abstraction
- application

Yet it is already computationally universal.

Lambda Calculus: The Three Term Forms

Variable

`x`

A name for a value

Abstraction

`λx. e`

A function definition

Application

`(f a)`

Apply a function to an argument

The important point is structural:

- variables do not compute
- abstractions wait for input
- applications are where computation happens

Beta Reduction

```
(λx. e) v → e[v/x]
```

Apply the function by substituting the argument for the bound variable.

```
(λx. x) 5 → 5

(λx. x + 1) 4 → 5

(λx. λy. x) 3 7 → (λy. 3) 7 → 3
```

A Worked Reduction

```
(λf. λx. f (f x)) (λz. z + 1) 3
```

```
(λx. (λz. z + 1) ((λz. z + 1) x)) 3

→ (λz. z + 1) ((λz. z + 1) 3)

→ (λz. z + 1) 4

→ 5
```

```
#check (fun (f : Nat → Nat) ⇒ fun (x : Nat) ⇒ f (f x))
#eval ((fun (f : Nat → Nat) ⇒ fun (x : Nat) ⇒ f (f x))
  (fun z : Nat ⇒ z + 1) 3)
```

Open in live.lean-lang.org

This is computation with nothing but functions and substitution.

Alpha Equivalence and Free Variables

```
λx. x ≡α λy. y
```

Renaming a bound variable does not change the meaning of the term.

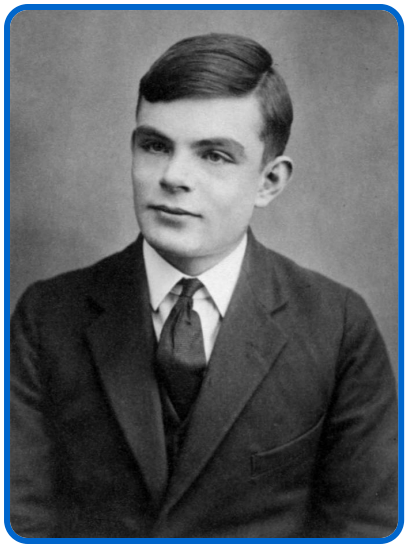
```
variable (y : Nat)

def f := fun x ⇒ x^2 + 1
def g := fun y ⇒ y^2 + 1

variable (x y : Nat)
abbrev F := fun z : Nat ⇒ z^2 + y
example : F x = fun z ⇒ z^2 + x := rfl
```

Open in live.lean-lang.org

Alan Turing and the Turing Machine



Alan Turing (1912–1954)

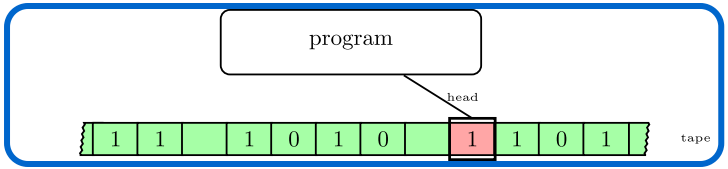
- developed the Turing machine model
- presented computation as state evolution on memory
- gave a machine-theoretic route to undecidability

Church and Turing gave two very different-looking models:

- lambda calculus = symbolic reduction
- Turing machine = local machine steps on a tape

These models were later understood to capture the same class of effective procedures.

The Turing Machine Picture



A Turing machine has:

- a tape divided into cells
- a head that reads and writes symbols
- a finite set of states
- a transition rule for each state-symbol pair

Each computation step is local:

1. read a symbol
2. write a symbol
3. move left or right
4. change state

Hilbert, the Decision Problem, and the Halting Problem

David Hilbert (1862–1943)

Hilbert's program asked whether mathematics could be made fully precise and mechanically tractable.

One central question was the **Entscheidungsproblem**:

Is there a general procedure that always decides whether a formal statement is provable?

Turing's halting problem gives a negative answer in machine form:

there is no single algorithm that always decides whether an arbitrary program will stop.

A First Surprise: Data as Behavior

In untyped lambda calculus, even familiar data can be represented by what it does.

Church booleans

```
true = λt. λf. t false = λt. λf. f

if b then e1 else e2 = b e1 e2
```

`true` picks the first branch.
`false` picks the second branch.

```
abbrev CBool := {α : Type} → α → α → α
def ctrue : CBool := fun {_} t f ⇒ t
def cfalse : CBool := fun {_} t f ⇒ f
def cIte (b : CBool) (t e : α) : α := b t e
```

Church Numerals

```
0 = λf. λx. x -- apply f zero times 1 = λf. λx. f x -- apply f once 2 = λf. λx. f (f x) -- apply f
twice n = λf. λx. f^n(x)
```

The point is not to compute arithmetic by hand here.
The point is conceptual:

- if functions are primitive,
- then booleans and numbers can still be encoded.

The Problem: Unrestricted Self-Application

```
Ω = (λx. x x) (λx. x x)
```

This is the standard bad example in untyped lambda calculus.

```
(λx. x x) (λx. x x) → ((λx. x x) (λx. x x))
```

After one beta-reduction step, the term becomes exactly itself again.
So the reduction repeats forever.

Why Add Types?

Untyped lambda calculus is powerful, but it does not by itself distinguish:

- meaningful application
- accidental nonsense
- unstable self-application

Typed lambda calculus adds discipline:

- applications must type-check
- malformed terms are rejected
- the system starts to support mathematical interpretation

Part II

Types and Discipline

From Programming Types to Mathematical Types

C

```
int x = 42;
int y = "hello"; // error
```

Lean

```
#check 42
#check "hello"
#check (42 + "hi")
```

Open in live.lean-lang.org

Programming intuition: types reject nonsense in computation.

Typed Existence: `a : A`

In type theory, the primitive form is

`a : A`

Set-theoretic picture

- first: an object `a`
- then: a set `A`
- then ask whether `a ∈ A`

Type-theoretic picture

- an object is introduced already typed
- `a : A` is a judgment
- there is no prior untyped background universe

Judgment vs Proposition

Judgments

- `42 : Nat`
- `true : Bool`
- `[1,2,3] : List Nat`

Propositions

- `a ∈ S`
- may be true or false

`π : Nat` is not a false proposition.
It is ill-typed syntax.

Simply Typed Lambda Calculus

STLC adds types to lambda terms.

```
x x -- not disciplined (λx. x x) (λx. x x) -- loops forever
```

With types:

- variables get types
- functions have types `A → B`
- applications are allowed only when types match

STLC: The Two Key Typing Rules

Abstraction

If `x : A` and `e : B`, then

`λx. e : A → B`

Application

If `f : A → B` and `a : A`, then

`f a : B`

Function Composition in Lean

```
variable {A B C : Type}
variable (F : A → B) (G : B → C) (a : A)

#check G (F a)
#check G ∘ F
```

Open in live.lean-lang.org

This is the computational side of implication:

- functions can be composed
- reasoning can be chained

Conditionals in a Typed Setting

```
#check ite
#check ite true
#check ite true 10 0
#eval ite false 10 20

def myIte (b : Bool) (t e : α) : α :=
  match b with
  | true ⇒ t
  | false ⇒ e

#eval myIte true 10 20
```

Open in live.lean-lang.org

After typed structure is in place, branching is no longer mysterious: it is just a typed operation.

Curry–Howard: First Glimpse

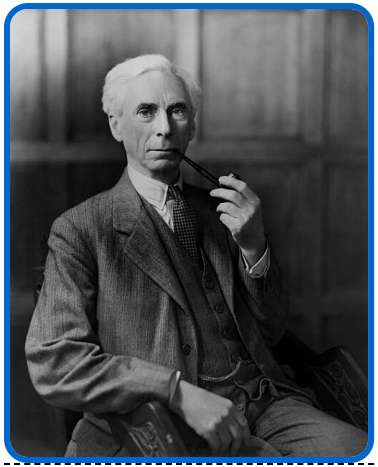
Once terms are typed, logic and computation begin to line up:

- proposition ↔ type
- proof ↔ term
- implication ↔ function

```
def andSwap (h : A ∧ B) : B ∧ A :=
 ⟨h.2, h.1⟩
```

The proof is a program term, and type checking verifies it.

Russell's Paradox and Type-Theoretic Discipline



Bertrand Russell (1872–1970)

Naive set formation leads to self-reference:

$R = \{x \mid x \notin x\}$

If $R \in R$, then $R \notin R$.
If $R \notin R$, then $R \in R$.

This is not an infinite loop.
It is a contradiction.

Why Types Help with Russell-Style Self-Reference

```
#check Nat
#check Type
#check Type 1
```

Open in live.lean-lang.org

Universe hierarchy gives stratification:

- `Nat : Type`
- `Type : Type 1`
- `Type 1 : Type 2`

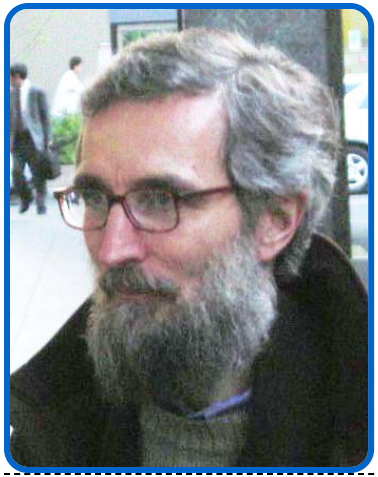
The benefit is conceptual:

- unrestricted self-application causes instability
- unrestricted self-reference causes contradiction

Part III

Dependent Types

Per Martin-Löf



Per Martin-Löf (b. 1942)

- developed intuitionistic type theory
- made it natural to let types depend on values
- directly influenced the style of reasoning used in Lean

Dependent types are the step from:

`fixed output type`

to

`output type depends on the input`

Why Simple Types Are Not Enough

Simple function types look like:

`A → B`

The output type `B` is fixed in advance.

But mathematics often needs richer information:

- lists of length exactly `n`
- safe indices below `n`
- predicates depending on a particular input

Dependent Types: Families of Types

A dependent type is a type depending on a value.

`Vector : Type → Nat → Type`

So these are different types:

- `Vector String 0`
- `Vector String 1`
- `Vector String 3`

```
import Mathlib

#check Vector
#check Vector String
#check Vector String 0
#check Vector String 1
```

Π Types and ∀

`Π (x : A), B(x)` is the type of dependent functions.

When the codomain is a proposition, Lean writes this as `∀`.

```
def noneg := Π x : Nat, x ≥ 0
def noneg' := ∀ x : Nat, x ≥ 0
example : noneg = noneg' := rfl
```

Open in live.lean-lang.org

Sum Types in Lean: `Sum` (Type) vs `Or` (Prop)

"Left case, or right case" – the **non-dependent choice**. Lean provides it in **two universes**, and they are different constructs:

In `Type`: `Sum α β` / `α ⊕ β`

```
#check Sum   -- Type → Type → Type
example : Nat ⊕ String := .inl 5
example : Nat ⊕ String := .inr "hi"
```

Carries data; can hold arbitrary types.

In `Prop`: `Or p q` / `p ∨ q`

```
#check Or    -- Prop → Prop → Prop
example (A B : Prop) (a : A) : A ∨ B := .inl a
example (A B : Prop) (b : B) : A ∨ B := .inr b
```

Proof-irrelevant disjunction.

Same shape, two universes. The **dependent** versions are parallel:
Σ (dependent pair in `Type`) and $\exists$ (dependent existential in `Prop`) – coming up next.

Prop vs Type: Proof Irrelevance vs Injectivity

```lean
variable (P Q : Prop)

-- Prop universe: two different constructions of (P ∧ Q) → (P ∨ Q)
def proof1 (h : P ∧ Q) : P ∨ Q := by
  have Pholds : P := h.1; exact Or.inl Pholds      -- picks LEFT
def proof2 (h : P ∧ Q) : P ∨ Q := by
  have Qholds : Q := h.2; exact Or.inr Qholds      -- picks RIGHT

example : proof1 = proof2 := rfl                   -- ✅ in Prop, all proofs equal

-- Type universe: Sum constructors are INJECTIVE
def sum1 : Nat ⊕ Nat := Sum.inl 1
def sum2 : Nat ⊕ Nat := Sum.inr 1
example : sum2 ≠ sum1 := Sum.inr_ne_inl             -- ✅ genuinely distinct values
```

▶ Open in live.lean-lang.org

In `Prop` **:** all proofs of the same proposition are *definitionally equal* (proof irrelevance).
In `Type` **:** `Sum.inl` and `Sum.inr` are *distinct constructors* – their values are never equal.

Σ Types – Dependent and Non-Dependent

```
-- Non-dependent: second component's type does NOT depend on the first
def sum3 : Σ x : Nat, Nat   := ⟨1, 1⟩

-- Dependent: Fin x mentions the first component x
def sum4 : Σ x : Nat, Fin x := ⟨4, 3⟩    -- 3 : Fin 4, so this type-checks

#eval sum4.1     -- 4
#eval sum4.2     -- 3                    (an element of Fin 4)
#check sum4.2    -- @Fin.mk sum4.1 ...   ← the TYPE mentions sum4.1 !
#print sum4
```

▶ Open in live.lean-lang.org

The type of `sum4.2` is `Fin sum4.1` – it literally mentions the first component. That's what "**dependent**" means in "dependent pair".

Subtype – Data + Prop Predicate

Subtype `{x : α // p x}` – a value of `α` paired with a **proof** that `p x` holds.

```
-- "natural numbers greater than 5"
def six   : {x : Nat // x > 5} := ⟨6, by decide⟩
def seven : {x : Nat // x > 5} := ⟨7, by decide⟩

-- Subtype has TWO fields: val (data) and property (proof)
#eval  six.val         -- 6                 (data, computable)
#check six.property    -- six.val > 5       (proof, lives in Prop)
```

Subtype sits **between** `Σ` and `∃`:

- Like `Σ`: carries data, you can `.val` to recover the witness
- Like `∃`: the second field is a `Prop`, so proof-irrelevant
- Used everywhere in Mathlib: `{n : ℕ // Prime n}`, `{f // Continuous f}`, …

▶ Open in live.lean-lang.org

Subtype Equality: Determined by `val`

Because the `property` field is **proof-irrelevant**, two Subtype values are equal iff their `val` s are equal.

```
example : six ≠ seven := by
  intro h
  -- If six = seven, then extracting .val gives six.val = seven.val
  have h_val : six.val = seven.val := congr_arg Subtype.val h
  -- But six.val = 6 and seven.val = 7, so 6 = 7 — contradiction
  contradiction
```

▶ Open in live.lean-lang.org

The three-way pattern:

- $\Sigma x,\ B(x)$ (pair of Type+Type) – both fields distinguishable
- **Subtype** `{x // p x}` (Type + Prop) – equal iff `val` s are equal
- $\exists x,\ p(x)$ (all in Prop) – any two proofs are equal

Σ Types and Ordinary Mathematics

`Σ (x : A), B(x)` is a dependent pair:

- a witness
- together with evidence about that witness

```
example : Σ n : Nat, n ≥ 5 := ⟨5, Nat.le_refl 5⟩
--                              ↑         ↑
--                           witness   evidence about the witness
```

Subtle point: $\exists x, P(x)$ is the *propositional quotient* of $\Sigma x, P(x)$ – Σ remembers the witness; $\exists$ may hide it. We will return to this in Week 3.

Inductive Types Build Mathematics

```
inductive Nat where
  | zero : Nat
  | succ : Nat → Nat

example : Nat.succ 0 = 1 := rfl
```

Open in live.lean-lang.org

Inductive definitions give:

- constructors for data
- elimination / recursion principles
- induction built into the type itself

Induction in Lean

```
theorem add_zero (n : Nat) : n + 0 = n := by
 induction n with
 | zero ⇒ rfl
 | succ n ih ⇒ simp [Nat.succ_add, ih]
```

Open in live.lean-lang.org

This is the Week 2 endpoint:

- data by constructors
- proofs by induction
- mathematics and computation inside one language

Week 3 Preview

Week 3 will continue with:

- identity types
- equality as paths
- path induction and transport
- the first real entry point into HoTT

Week 2 stops deliberately before this material, so that computation, types, and dependent structure remain clear.

Homework

1. Review the lambda-calculus reduction examples from today's handout.
2. Explain in words why `Ω = (λx. x x)(λx. x x)` never reaches normal form.
3. Review `Π`, `Σ`, `Vector`, and the induction example in Lean.
4. Read the Week 3 preview topics before the next lecture.

See You Next Week

Next topic: identity types, paths, and the HoTT viewpoint